SYNTHOLOGY: Ontology-Based Data Generation for Neurosymbolic Reasoning

Vincent VAN SCHEPENDOM [1], Cas PROOST and Pieter BONTE
Department of Computer Science, KU Leuven campus Kulak Kortrijk

Abstract. Symbolic reasoners offer a simple contract: provide an ontology, receive entailments. Neurosymbolic reasoners promise the same task with added scalability and tolerance to noise, but break this contract by requiring an ontology-specific training dataset, which in most cases does not exist and must be gathered from scratch. This bottleneck blocks practitioners from applying neurosymbolic reasoners to new ontologies. We present SYNTHOLOGY, an ontology-agnostic synthetic data generator that closes this gap by producing the required training data from any OWL 2 RL ontology, with no need for manual data gathering. SYNTHOLOGY engineers data through ontology-guided backward-chaining proof search, which guarantees deep multi-hop derivations by construction, and introduces proof-based negative sampling. Together, these properties force models to traverse the full reasoning chain to correctly classify triples. Our evaluation shows that a model trained on SYNTHOLOGY data outperforms models trained on baseline data, with a particularly robust advantage in rejecting near-miss negatives. Proof-based negative sampling yields consistent gains over random and constrained sampling on all learning-quality metrics, and SYNTHOLOGY scales to ontologies on which traditional (ontology specific) generators run out of memory. To our knowledge, SYNTHOLOGY is the first ontology-agnostic synthetic data generator that provides multi-hop coverage, near-miss negatives, and derivation metadata by construction, a step toward making neurosymbolic reasoners as readily deployable on a new ontology as their symbolic counterparts.

Keywords. Neurosymbolic AI, Reasoning, Data Generation, Negative Sampling

1. Introduction

Knowledge graphs (KGs) and ontologies form the backbone of structured knowledge representation on the Semantic Web. The task of deriving new, implicit knowledge from these structures is known as *knowledge graph reasoning* (KGR) [1,2], and is the broader task this work is ultimately concerned with. Traditionally, KGR is performed by symbolic reasoners [3,4], which directly execute the ontology's rules with perfect soundness and completeness. Symbolic reasoners offer a simple contract: provide an ontology, receive entailments. While elegant, this approach hits a practical wall: symbolic reasoners are sensitive to noise and computationally expensive, making them infeasible on massive or messy real-world KGs [5].

Neurosymbolic reasoning has emerged as the proposed alternative within the field of KGR [6]. Instead of *executing* the ontology's rules at inference time, a neural model

[1]Corresponding Author: Vincent Van Schependom, vincent.vanschependom@student.kuleuven.be

is trained to *imitate* them. The resulting behavior is called *approximate reasoning*: the model predicts the entailments of the underlying knowledge base in a way that is scalable and tolerant to noise [7]. This shift, however, breaks the symbolic-reasoner contract. A symbolic reasoner operates directly on an ontology and immediately produces entailments, whereas a neurosymbolic reasoner first has to be *trained* on a dataset that reflects the target ontology. For most ontologies, such a dataset does not exist and must be gathered from scratch, and this bottleneck blocks practitioners from applying neurosymbolic reasoners to new ontologies at all.

Could existing data sources fill this gap? Real-world datasets such as DBpedia or Freebase [8,9] are noisy, incomplete, and tied to specific domains, so they neither cover arbitrary ontologies nor support training a model to learn genuine logical reasoning instead of statistical shortcuts. Synthetic data generators could in principle solve this, but standard approaches typically use procedural generation of base facts [10,11], followed by materialization: a logical reasoner infers the to-be-predicted targets from the previously generated base facts, as in [12]. This methodology often produces structurally shallow graphs, because the generation is not ontology-aware. The resulting graphs contain a lot of trivial facts, while the deep, multi-hop derivations that the model is actually meant to learn occur only by accident. On top of this, the resulting datasets lack challenging negative samples, which are critical for training neurosymbolic models: trivial negatives are easy to reject from surface features alone [13], so the training signal tends to collapse to pattern matching rather than reasoning.

To address this gap, we present SYNTHOLOGY, an ontology-driven synthetic data generator. Our novel generator accepts *any* ontology expressed in the supported subset of OWL 2 RL. In a backward-chaining manner, it purposefully engineers training data that inherently requires multiple hops to be correctly predicted. Crucially, it also generates near-miss negative samples based on the constructed proof trees. These negatives *look* plausible, but are not logically entailed, forcing models to learn the underlying ontology structure. In summary, our main contributions are:

- **Synthology**: an ontology-agnostic data generator that, given *any* OWL 2 RL ontology, produces a training set sufficient to train a neurosymbolic reasoner over that ontology. This removes the per-ontology data-gathering step that currently blocks deployment of neurosymbolic reasoners on new ontologies.
- **Backward-chaining proof construction** the core mechanism, which guarantees deep multi-hop coverage by construction rather than incidentally.
- **Proof-based negative sampling**: a method for constructing negative samples based on proof trees that produce near-misses, forcing models to traverse the full reasoning chain rather than rely on surface features.
- **Empirical evaluation**: a comparative study across two ontologies (family tree, OWL2Bench) demonstrating significant advantages in hop distribution, predicate coverage, negative sample quality, and scalability over existing Unguided Deductive Materialization methods.

We also release an open-source (optimized) reimplementation of the Recursive Reasoning Network [14] (original paper did not provide the code), which is used as the evaluation model throughout this paper.

Section 2 introduces the necessary background, Section 3 reviews related work, Section 4 details SYNTHOLOGY's design, Section 5 describes the experimental setup, Section 6 reports results, and Section 7 concludes the paper and outlines future work.

2. Background

We first define knowledge graphs and ontologies, then describe what we mean by approximate reasoning in a neurosymbolic setting, and finally explain the role of negative sampling in training neural reasoners.

2.1. Knowledge Graphs and Ontologies

We consider a knowledge graph as a collection $\mathcal{G}$ of concrete facts (assertions) about specific entities, referred to as the ABox. To provide semantics and enable reasoning, the KG is paired with an ontology $\mathcal{O}$ (the TBox). Together, the factual KG and the ontology form a knowledge base $\text{KB} = \langle \mathcal{G}, \mathcal{O} \rangle$.

For example, the triples $\langle \text{Alice}, \text{hasParent}, \text{Bob} \rangle, \langle \text{Bob}, \text{hasParent}, \text{Charlie} \rangle$ are explicit, *base facts* in the KG, while a rule like $\langle X, \text{hasParent}, Y \rangle \wedge \langle Y, \text{hasParent}, Z \rangle \Rightarrow \langle X, \text{hasGrandparent}, Z \rangle$ is part of the ontology $\mathcal{O}$. Both are present in the KB. Combining them, one can derive implicit facts, like $\langle \text{Alice}, \text{hasGrandparent}, \text{Charlie} \rangle$. These *inferred facts* are not explicitly stated in the ABox, but logically entailed by the ontology given the existing *base facts*. The *base/inferred* terminology will be used throughout this paper.

In this work, we focus on the OWL 2 RL profile [15] as the target rule language, though SYNTHOLOGY is built for extensibility to other profiles.

2.2. Neurosymbolic Reasoning

Predicting which implicit facts are entailed by a given KB without explicitly applying the ontology's rules is a form of knowledge graph reasoning (KGR). Previous work refers to this as *approximate reasoning*, trading strict logical completeness for scalable prediction of entailed facts [7]. Standard knowledge graph embeddings (KGEs), such as TransE [16], excel at capturing single-hop patterns through vector-space geometry, but struggle with the deep, multi-hop logical chains [1,14]. For approximate reasoning tasks, this motivates the shift toward neurosymbolic architectures, which combine the scalability of neural networks with the logical precision of symbolic reasoning [2]. A promising example is the *Recursive Reasoning Network* (RRN) [14], which will serve as the evaluation model throughout this paper. The RRN is trained to predict the likelihood $\mathbb{P}(\langle s, p, o \rangle)$ of a triple being logically entailed by the provided base facts. It uses an iterative message-passing mechanism, allowing the model to capture complex, multi-hop logical dependencies. Crucially, the training data must contain sufficient evidence of these deep patterns, and this is precisely what SYNTHOLOGY is designed to provide.

2.3. Negative Sampling

Apart from deep chains, another critical factor in training neurosymbolic reasoners (like the RRN) is negative sampling. Since KGs contain only positive facts, models must learn to distinguish true inferences from false ones by training against corrupted (and thus false) facts. We consider two standard corruption strategies: *random* corruption replaces the subject or object with an arbitrary individual, while *constrained* corruption additionally requires the replacement to satisfy the corrupted predicate's declared domain or range. Recent reviews of negative sampling strategies in KGR emphasize that creating 'hard' negatives – structurally similar, plausible facts that are *not* logically entailed – is

a critical element in training neurosymbolic models, as it forces them to learn *nuanced* logical patterns rather than relying on superficial indicators [13].

3. Related Work

This section surveys existing approaches to generating training data for KGR. We organize them by methodology and, for each family, ask the same question: can it deliver a training set for *any* ontology without per-ontology data engineering?

3.1. Benchmark-Based ABox Generators

Several benchmarks for OWL reasoning provide a procedural ABox generator that creates synthetic data following a fixed schema. The Lehigh University Benchmark (LUBM) [10] populates a university domain, including students, professors, courses, and departments, at configurable scales. Its successor, the University Ontology Benchmark (UOBM) [11], extends LUBM to include additional OWL constructs. OWL2Bench [17], finally, provides a generator paired with ontologies that cover all four OWL 2 profiles (including RL), once again over a university domain. The problem is that each of these generators is hand-built around its target ontology: the type distributions, individual roles, and relation densities are tuned to a specific schema. None of them can be reused on a new ontology without significant manual re-engineering, which is exactly the bottleneck a deployable neurosymbolic pipeline needs to eliminate.

3.2. Unguided Deductive Materialization

To produce training targets, such a benchmark generator (or a fully random ABox) can be paired with a symbolic reasoner that materializes the deductive closure [12]. We refer to this two-step pipeline as *Unguided Deductive Materialization* (UDM): a procedurally or probabilistically generated set of base facts is first constructed (either tied to a fixed ontology or drawn at random), and a symbolic reasoner then materializes all logically entailed facts to form the training targets. Negatives are typically added via uniform random subject or object corruption [16].

The issue with UDM is that generation *is not guided by the ontology*. The result is a training distribution shaped by what is easy to generate rather than what the model needs to learn. The generator first produces individuals and assertions from statistical templates, and the reasoner is invoked only afterwards. As a result, the generated data rarely satisfies the conditions required to fire long rule chains: predicates that depend on the simultaneous presence of multiple different facts produce few, if any, inferences. On the other hand, shallow schema propagations (like `rdf:type` and `rdfs:subClassOf`) tend to dominate the closure. Facts that require *multi-hop reasoning* are therefore systematically underrepresented in the resulting training data, regardless of how many base facts are generated, and neural models trained on this output are prone to *shortcut learning* [18].

A second concern is scalability. Forward-chaining reasoners like Apache Jena [4] typically use semi-naive evaluation, which iteratively saturates the ABox until no new facts can be derived. Although each iteration is bounded, the materialized closure must be retained in memory throughout. In practice, this causes materializers to fail at

production-scale ontologies. We refer to this ceiling as the *reasoning wall* and we characterize it empirically in Section 6.3.

3.3. Ontology-Specific ASP Generators

Rather than relying on a benchmark generator + reasoner pipeline, some prior work uses *answer set programming* (ASP). The most prominent example is the RRN paper [14], which provides an ASP program tailored to a single ontology (family tree). While this couples generation to logical structure (unlike UDM), the ASP program must be hand-written for each new ontology, making extension to a different domain non-trivial and reintroducing the per-ontology engineering cost that we are trying to eliminate.

A separate weakness of this approach is its handling of negatives, which relies on *exhaustive* computation, rather than *strategic* sampling. ASP natively supports negation constructs, allowing the generator to simply compute *all* possible inferences for every generated sample. While this guarantees the presence of negative examples, it produces heavily skewed datasets where the vast majority of samples are trivial negatives.

3.4. LLM-Based Generators

Large language models (LLMs) have recently been explored for KG construction, generating triples from natural-language descriptions of a domain. While promising for naturalistic verbalization, LLM outputs lack formal correctness guarantees and are thus not always suited for generating the logic-consistent data on which a neurosymbolic reasoner can be supervised [19]. Furthermore, LLM-based generators do not produce the derivation structure (proof trees, hop depths) that is required to construct hard negatives or to verify multi-hop coverage. They generate triples, not the reasoning that supports them.

Across these existing families of approaches, three shortcomings recur. (i) Generation is not ontology-aware, so multi-hop inferences appear only by chance and shallow schema propagations dominate. (ii) Negative samples are constructed in isolation from the derivation that produced the positive, yielding trivially rejectable corruptions rather than near-misses that demand genuine reasoning. (iii) Forward-chaining materialization scales poorly on production-sized ontologies. Together, these shortcomings explain why no existing pipeline lets a practitioner go from "I have an ontology" to "I have a trained neurosymbolic reasoner" without significant per-ontology data engineering.

4. Methodology: Targeted Data Generation

Existing pipelines force a practitioner to choose between a hand-built generator for a specific ontology and an ontology-blind "generate + materialize" approach whose output is shallow and full of trivial negatives. Neither restores the symbolic-reasoner contract. To address this, we introduce SYNTHOLOGY. Unlike standard pipelines, our generator engineers an ABox through ontology-guided backward-chaining proof search. In this way, we produce base facts driven by logical *necessity* rather than statistical randomness. The full codebase, including the generator, ontologies, and experimental setups, is open-source[2] under the MIT license.

[2] `https://github.com/schependom/synthology`

SYNTHOLOGY accepts ontologies expressed in a supported subset of OWL 2 RL[3]. The subset covers the constructs that drive multi-hop inference chains in practice (subclass, subproperty, domain/range, inverse, property chains, symmetry, transitivity); other OWL 2 RL constructs can be added by extending the parser, without changes to the chainer. SYNTHOLOGY executes a three-phase pipeline. A *parsing* phase first compiles the ontology into a set of executable Horn rules $\mathcal{R}$, a schema $\mathcal{S}$ (classes, relations), and constraints $\mathcal{C}$. The *generation* phase (Algorithm 1) then executes a sample-level loop, in which backward-chaining proof search constructs proof trees from grounded rule conclusions. Finally, in the *validation* phase, each candidate graph is checked against $\mathcal{C}$ and injected with negative samples (Section 4.3).

The pipeline is modular by design: new OWL constructs can be supported by adding parser handlers and rule templates, without changing the chainer itself. Users can trade structural properties against generation cost via the configurations introduced in the rest of this section (π_{reuse}, the recursion cap, the depth-bucket weights, etc.), and the framework supports *sweeps* for systematic optimization of these parameters.

4.1. Sample Construction

Data construction (Algorithm 1) assembles one knowledge graph per iteration. A random subset of rules $\mathcal{R}' \subseteq \mathcal{R}$ is selected. For each selected rule $r \in \mathcal{R}'$, k_r independent groundings of its head are created and passed to the backward chainer, which returns a set of proof trees P_r. We therefore refer to the k_r independent groundings of the rule head as *proof roots*. A configurable number of proofs is chosen per rule and merged into a global *proof map* M. This map links every atom appearing in the selected proofs with the list of derivations that produced it. M is then materialized into a candidate KG, completed with class memberships implied by the domain and range declarations of its triples (*schema completion*). This candidate KG is then validated and injected with negative samples.

We have to be careful, since the proof search space grows combinatorially: each premise may admit multiple sub-proofs, and combining them requires a Cartesian product. This repeats recursively down the tree. SYNTHOLOGY therefore uses *bounded* proof engineering: the global proof depth, the number of times a recursive rule may be reapplied, and the number of proof trees retained per atom are all capped.

A key mechanism for producing connected, non-trivial graphs is the *individual reuse pool*. When the chainer binds an unbound variable, it reuses an existing pool member with probability π_{reuse}. If this reused individual violates domain/range constraints, a fresh individual is created and added to the pool. This controlled individual sharing causes derivations to overlap, resulting in interconnected graphs rather than isolated proof trees. The reuse probability is exposed as a configurable hyperparameter, allowing users to trade graph density against derivation diversity for their target ontology.

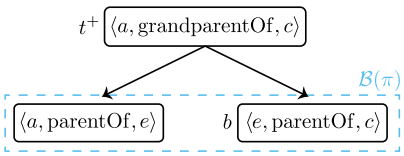
Because each iteration of the outer loop generates one knowledge graph independently, the memory footprint of generation is bounded by a single sample rather than by the cumulative deductive closure. This is the structural property that lets SYNTHOLOGY scale where forward-chaining materializers cannot.

[3]RDFS schema axioms (`rdfs:subClassOf`, `rdfs:subPropertyOf`, `rdfs:domain`, `rdfs:range`), OWL 2 property axioms `owl:inverseOf`, `owl:propertyChainAxiom`, `owl:SymmetricProperty`, `owl:TransitiveProperty`, and `owl:ReflexiveProperty`, and constraints `owl:IrreflexiveProperty`, `owl:AsymmetricProperty`, `owl:FunctionalProperty`, and `owl:disjointWith`

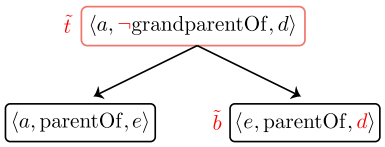
Algorithm 1 Sample Construction

Require: Rules $\mathcal{R}$, chainer $\mathcal{B}$, target count N_S, configuration Θ
Ensure: Sample set $\mathcal{D}_S$

$\mathcal{D}_S \leftarrow \emptyset$
while $|\mathcal{D}_S| < N_S$ and attempt budget not exhausted **do**
  Reset proof map $M \leftarrow \emptyset$ and individual pool
  Select rule subset $\mathcal{R}' \subseteq \mathcal{R}$
  for each rule $r \in \mathcal{R}'$ **do**
    Sample root count $k_r \sim \mathcal{U}[k_{\min}, k_{\max}]$
    $P_r \leftarrow \bigcup_{i=1}^{k_r} \textsc{Prove}(g_i, 0, \emptyset)$, with g_i a fresh grounding of r's head
    Merge up to n_r proofs from P_r into M
  end for
  if $M = \emptyset$ **then continue**
  end if
  Convert M to a candidate KG; apply schema completion
  Perform validation checks
  Add negatives using the configured strategy (Section 4.3)
  $\mathcal{D}_S \leftarrow \mathcal{D}_S \cup \{\text{KG}\}$
end while



(a) A proof tree π with base facts $\mathcal{B}(\pi) = \{\langle a, \text{parentOf}, e\rangle, \langle e, \text{parentOf}, c\rangle\}$, which supports the inferred fact $t^+ = \langle a, \text{grandparentOf}, c\rangle$.



(b) Corrupting a base fact $b \in \mathcal{B}$ to $\tilde{b}$ by switching the object $c \to d$ in π yields the near-miss negative $\tilde{t} = \langle a, \neg\text{grandparentOf}, d\rangle$.

Figure 1. Proof-based corruption in action.

4.2. Backward-Chaining Proof Search

For each of the candidate rules $p_1 \wedge ... \wedge p_n \to h$, the chainer first checks the per-rule recursion cap, and it prunes the branch if the rule's head h is the inverse of the parent predicate (to eliminate infinite inverse loops). Then, the chainer unifies the goal g with h, binds remaining variables (via the reuse pool), and recursively proves the grounded premises $p_1, ..., p_n$. At most $P_{\max}$ combinations of premise proofs are retained per goal. If no rule succeeds, the chainer falls back to a base proof, ensuring every goal predicate has at least one derivation. To counteract the dominance of shallow schema proofs, SYNTHOLOGY supports *depth-stratified sampling*, which distributes retained proofs across easy ($d \leq 1$), medium ($2 \leq d \leq 4$), and hard ($d \geq 5$) buckets using normalized weights. An optional *signature sampling* step additionally groups proofs by their structural signature (the multiset of rule names in the tree) and keeps one representative per signature. This reduces near-isomorphic proofs. The resulting dataset has a strict base-versus-inferred fact distinction, together with derivation metadata for each triple.

4.3. Proof-Based Negative Sampling

Random and constrained corruption (Section 2.3) both operate on a single triple in isolation and are therefore blind to the derivation that produced it. While current negative sampling literature typically categorizes methodologies into static, dynamic, or purely adversarial approaches [13], SYNTHOLOGY introduces *proof-based* negative sampling, which targets *the logical support* of inferred facts. It does this by corrupting a base fact and propagating the error upward along its proof tree.

Let a positive target triple t^+ have at least one proof tree π, and let $\mathcal{B}(\pi)$ be the base facts used to derive t^+. The proof-based strategy samples a base atom $b \in \mathcal{B}(\pi)$ and perturbs its subject or object, yielding $\tilde{b}$. We note that $\tilde{b}$ is *not* added to the input KG; it exists only as a *conceptual* corruption for generating the negative target. The subject or object substitution is then propagated through the derivation chain, producing a logically flawed goal atom $\tilde{t}$. This triple is only accepted as a negative if it does not coincide with an existing positive triple in the graph.

Consider the rule $\langle X, \text{parentOf}, Y\rangle \wedge \langle Y, \text{parentOf}, Z\rangle \Rightarrow \langle X, \text{grandparentOf}, Z\rangle$ and base facts $\langle a, \text{parentOf}, e\rangle, \langle e, \text{parentOf}, c\rangle$, for example. Figure 1 shows the proof tree π that derives the inferred fact $t^+ = \langle a, \text{grandparentOf}, c\rangle$. If we corrupt the object of the second premise, we obtain $\tilde{b} = \langle e, \text{parentOf}, d\rangle$. Since the variable binding $Y = e$ remains intact, the rule still fires and propagates the error upward to $\tilde{t} = \langle a, \neg\text{grandparentOf}, d\rangle$. The resulting false conclusion $\tilde{t}$ would be backed by a structurally perfect proof, except that $\langle e, \text{parentOf}, d\rangle$ does not exist in the input KG.

We now illustrate how a model should make a decision about $\tilde{t}$. Because $\tilde{b}$ does not appear in the actual KG, no valid proof of $\tilde{t}$ can exist within it. Yet, the proof structure itself remains sound, except for one corrupted leaf. Indeed, the derivation chain π would produce $\tilde{t}$ if $\tilde{b}$ were true. The model is thus trained to predict $\mathbb{P}(\tilde{t}) \approx 0$ in this case.

Our proof-based negative samples are *difficult* to learn: the model sees a structurally perfect proof, but must recognize that a single leaf premise is missing, therefore rejecting the conclusion. Learning to recognize and reject such near-miss negatives requires the model to traverse the full reasoning chain (and verify that each step is supported by the right facts), forcing *genuine* multi-hop reasoning rather than pattern matching.

To meet the configured negative ratio in finite time, the generator can fall back to constrained corruption when proof-based corruption fails. In practice, however, fallback rarely occurs for dense ontologies (such as our family tree ontology[4]), and the dataset consists almost entirely of hard proof-based negatives.

Finally, SYNTHOLOGY also provides a *mixed* corruption strategy, blending proof-based negatives with constrained corruptions in a user-specified ratio. The intuition is that simpler structural negatives could complement the learning of more complex proof-based ones. Whether this hypothesis holds in practice is examined in Experiment 1.

5. Experimental Setup

We evaluate SYNTHOLOGY along two axes. First, we measure properties of the generated data itself: hop distribution, predicate coverage, generation time, and memory footprint. Second, we measure the downstream effect on a neurosymbolic reasoner trained

[4] An OWL 2 RL translation of the `family-tree` ASP program from [14].

on that data, using a RRN [14] as the evaluation model. Three experiments test three claims: (i) Experiment 1 isolates the effect of proof-based negative sampling against random and constrained baselines on a fixed ontology; (ii) Experiment 2 compares full SYNTHOLOGY generation against an UDM baseline; (iii) Experiment 3 evaluates generalization to a richer benchmark ontology (OWL2Bench) and probes the scalability ceiling of forward-chaining materialization.

All RRN training experiments were conducted on a single NVIDIA Tesla V100 GPU (32 GB RAM). Dataset generation and scalability experiments were run on a CPU node of an LSF HPC cluster with an Intel Xeon Gold 6142 processor (2.60 GHz, 16 cores).

5.1. The Unguided Deductive Materialization (UDM) Baseline

We treat UDM as a family of approaches rather than a single fixed dataset: an ABox is first constructed under the constraints of the given input ontology, either (1) randomly or (2) procedurally (e.g. via a benchmark). After the ABox is generated in one of the two ways, a symbolic reasoner (in our case Apache Jena [4]) materializes its deductive closure to obtain positive targets. Negatives are then added via 1:1 random subject/object corruption applied uniformly to *all* positive triples.

For Experiment 2, we use the random ABox variant (1): we create an individual pool per sample, assign each individual a random set of ontology classes, and create object-property facts by uniformly sampling predicates (under the schema constraints) until the configured base-fact budget is reached. Experiment 3 uses the procedural variant (2), instantiating the ABox via the OWL2Bench Java generator. The Jena reasoning profile differs per experiment: Experiment 2 uses a lightweight profile focused on property-chain inferences (avoiding broad shallow derivations such as disjointness, in order to isolate deep multi-hop reasoning), while Experiment 3 uses a more expressive profile that supports the richer OWL 2 RL semantics required by the OWL2Bench ontology.

To ensure a *matched budget*, we balance both the base-fact count and the positive inferred-target count between the two pipelines. Both generators are configured to exceed the desired budget, after which a post-hoc balancing script trims samples in generation order until the cap is reached. In practice, the resulting budgets agree to within a small margin (see Table 3). Because Jena does not natively provide proof-depth distributions, we compute hop depth via a custom derivation-tracing pass over inferred facts, which introduces additional runtime overhead in Experiment 2 beyond closure computation itself (see Table 4).

For Experiment 3, this derivation logging is disabled during the full-scale baseline generation ($u = 14$ universities) due to its memory cost (Section 6.3). Instead, derivation logging is enabled only in a smaller run ($u = 3$ universities), whose hop distribution is used as a representative proxy for the full-scale run. This substitution is valid because OWL2Bench university ABoxes are independent fragments over the same TBox: the same rules fire at the same proof depths regardless of u.

All exact SYNTHOLOGY configurations and the full set of RRN hyperparameters used for each experiment are provided in the open-source repository.

5.2. The Evaluation Model: Recursive Reasoning Network

Because the original RRN publication did not provide an official source-code release, we reimplemented the architecture from scratch. To fully utilize GPU parallelization,

we grouped message passing updates into batches. The reimplementation preserves the original learning objective and matches the reference accuracy on the family tree dataset closely, while reducing training time drastically compared to the unbatched version.

The RRN outputs a probability score $P := \mathbb{P}(\langle s, p, o \rangle) \in [0, 1]$ indicating the likelihood that the target fact $\langle s, p, o \rangle$ is entailed by the base facts. We evaluate it as a binary classifier and report PR-AUC and AUC-ROC as threshold-independent metrics, alongside F1-score and the False Positive Rate (FPR) at threshold $P > 0.5$. We treat PR-AUC and FPR as the primary metrics, because the central claim of this paper concerns robustness against logically false near-miss negatives, which is a property that PR-AUC and FPR penalize more aggressively than AUC-ROC [20]. Per-class accuracy (Acc^+, Acc^-) is reported for completeness in Table 1, because it helps diagnose the conservatism or permissiveness of each model.

5.3. Experiment 1: Negative Sampling Strategy Evaluation

Using the family tree ontology, we generate SYNTHOLOGY variants that differ only in the negative-sampling strategy: standard random corruption, constrained random corruption, pure proof-based corruption, and a mixed strategy (blending 50% constrained and 50% proof-based corruption). Each variant consists of $N_{\text{train}} = 2000$ training, $N_{\text{val}} = 500$ validation, and $N_{\text{test}} = 500$ test knowledge graphs. A separate RRN is trained on each variant and evaluated on the same held-out test set, rich in near-miss targets.

5.4. Experiment 2: Multi-Hop Quality

To test multi-hop reasoning capability, we use the same family tree ontology, since it enables deep inference chains. An RRN trained on the UDM baseline (referred to as RRN_{UDM}) is compared against an RRN trained on SYNTHOLOGY data (referred to as RRN_{Syn}) under the matched-budget protocol described in Section 5.1. The sizes of the input graphs were balanced to have $\sim$200k input facts per dataset (see Table 3). Evaluation is performed on two test sets. On the one hand, we evaluate on a held-out test set, generated once by SYNTHOLOGY with elevated recursion/depth limits (reused for both models). On the other hand, to eliminate any potential bias from the SYNTHOLOGY-generated test set, we also evaluate on a test set generated by UDM.

We note that *completely* random ABox generation is deliberately the *worst case* for the UDM pipeline: the reasoner receives unstructured inputs, and this is therefore the hardest setting in which to produce deep inference chains. An ontology-aware ABox, constructed with handcrafted heuristics over the statistical distribution of types and relations, would close part of the gap, but every such heuristic is ontology-specific and is therefore not generalizable. Experiment 3 re-runs the comparison against a strong, benchmark-provided ABox generator to test the gap from the opposite direction.

5.5. Experiment 3: Generalization to Complex Ontologies

To demonstrate generalization to more complex ontologies, and to validate the ontology-agnostic claim of SYNTHOLOGY, we evaluate both generators (ours and the UDM approach) on the OWL2Bench benchmark ontology. For the UDM baseline, the OWL2Bench Java generator instantiates a university ABox for u universities, Apache Jena materializes its deductive closure, and the resulting graph is partitioned via BFS

Table 1. RRN predictive relational performance for Experiments 1 and 2. (†) = UDM-generated test set.

Exp.	Dataset	PR-AUC (↑)	AUC-ROC (↑)	FPR (↓)	F1 (↑)	Acc^{+}. (↑)	Acc^{-}. (↑)
1	Random	50.7%	0.770	**20.9%**	0.527	58.7%	**78.8%**
	Constrained	53.6%	0.830	37.7%	0.606	93.5%	62.1%
	Proof-based	**54.6%**	**0.843**	36.2%	**0.628**	**96.0%**	63.4%
	Mixed	46.6%	0.730	63.8%	0.490	94.9%	36.6%
2	UDM	38.8%	0.594	54.1%	0.457	69.7%	45.9%
	SYNTH.	**60.3%**	**0.825**	**34.6%**	**0.636**	**86.3%**	**65.1%**
2 †	UDM	41.8%	0.643	45.3%	0.483	63.3%	54.7%
	SYNTH.	42.9%	0.628	25.7%	0.419	41.2%	74.3%

Table 2. Per-hop RRN relational-triple performance on two test sets. Hop ≥ 3 is omitted for the UDM test set because Jena produced no negative targets at that depth. **Bold**: best per column within each test-set panel.

Test set	Method	Hops	Triple Acc. (↑)	F1 (↑)	FPR (↓)
SYNTHOLOGY (2)	UDM	1	42.1%	47.1%	76.9%
		2	69.4%	49.6%	22.6%
		≥ 3	58.5%	43.2%	40.7%
	SYNTH.	1	**81.5%**	**73.9%**	**21.4%**
		2	**74.9%**	**57.4%**	**17.0%**
		≥ 3	**79.5%**	**69.3%**	**20.5%**
UDM-generated (2 †)	UDM	1	57.1%	55.4%	45.3%
		2	**60.0%**	**59.3%**	44.3%
	SYNTH.	1	**65.4%**	**56.2%**	**22.3%**
		2	57.8%	42.6%	**24.5%**

(initialized at random individuals and capped at k individuals per subgraph) into sample-sized knowledge graphs. Inferred triples that span different subgraphs are discarded.

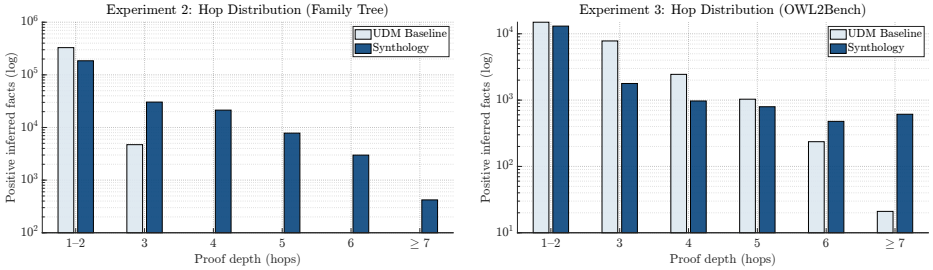
6. Results and Discussion

6.1. Experiment 1: Negative Sampling Strategy Evaluation

Table 1 reports RRN performance for the four negative-sampling strategies. Proof-based negative sampling achieves the best score on four of six metrics that capture genuine ranking and classification quality (PR-AUC, AUC-ROC, F1, and Acc^{+}) with consistent and substantial improvements over random corruption and smaller but reliable improvements over constrained corruption. Random corruption wins on FPR and negative triple accuracy (Acc^{-}),but reading them in isolation is misleading. The random-trained model achieves Acc^{-} of 78.8% but only 58.7% Acc^{+}. This reflects a classifier that learned *conservatism* rather than reasoning: it rejects most uncertain triples, which trivially produces a low FPR and a high Acc^{-}, but at the cost of missing a large fraction of true entailments. In contrast, both constrained and proof-based models exhibit the inverse profile: high positive accuracy ($\sim$93–96%) with moderate negative accuracy ($\sim$62–63%). This indicates that they successfully identified most true entailments, but occasionally accepted logically false near-misses, since these are hard to learn by design.

The mixed strategy performs worst overall, falling below even the random baseline on PR-AUC, AUC-ROC, and F1, and yielding a negative accuracy of only 36.6%. The culprit is training-signal inconsistency: within a single batch, the model encounters neg-

atives whose detection requires different reasoning (shallow type-compatibility checks *and* deep proof-chain tracing). This prevents a stable decision boundary from being established. Rather than benefiting from the complementary signals, the network oscillates between two incompatible objectives and converges to an aggressive positive bias. This bias is reflected in Table 1: the mixed strategy has a large false positive rate of 63.8%, the worst of all four strategies. A *blending schedule* may implement the complementary-signal method better than the naive 50/50 mix and is worth exploring in future work.



(a) Experiment 2: *family tree*. SYNTHOLOGY outperforms the UDM baseline, producing a deep hop distribution, in contrast with the baseline, which produces 99.3% 1–2 hop facts and only 4,716 facts with ≥ 3 hops.

(b) Experiment 3: *OWL2Bench*. SYNTHOLOGY creates a deeper hop distribution than the UDM baseline, which gets a 'head start' because of the sophisticated OWL2Bench ABox (Section 6.3).

Figure 2. Distribution of positive inference hop depths in the SYNTHOLOGY and UDM datasets for Experiment 2 (family tree) and Experiment 3 (OWL2Bench). The Experiment 3 UDM baseline distribution is measured from the 3-university run, used as a proxy because full-scale derivation logging is infeasible (Section 5.1).

6.2. Experiment 2: Multi-Hop Quality

Despite matched generation budgets, the UDM baseline and SYNTHOLOGY datasets show substantial differences in both reasoning complexity and structural composition. Table 3 summarizes the topological differences. Both methods produce comparable input-graph sizes (225k vs. 206k input facts, an 8.4% gap) and an almost identical total target volume ($\sim$640k). The UDM baseline produced a larger number of inferred targets (333k vs. 249k for SYNTHOLOGY). However, as illustrated in Figure 2a, this volume is deceptive: UDM produced an overwhelming majority of its inferences at depth $d \leq 2$, with fewer than 1% of its facts requiring paths of $d \geq 3$ (only 4,716 inferences). In contrast, SYNTHOLOGY engineered a robust distribution of deep multi-hop proofs, creating 60,310 inferences at $d \geq 3$ and reaching logical chains up to $d = 7$. UDM's higher inferred target count is thus simply the result of a combinatorial explosion of shallow, trivial inferences, whereas SYNTHOLOGY's budget is allocated towards deeper derivations.

Furthermore, the UDM baseline's unguided generation entirely failed to materialize inferences for 18 complex (e.g. `secondUncleOnceRemovedOf`) and gender-specific relations, resulting in exactly zero inferred targets for these predicates. This highlights a critical flaw in relying on unguided materialization: structurally demanding or highly constrained logical derivations occur only incidentally, making the UDM dataset systematically deficient. SYNTHOLOGY achieved complete predicate coverage, confirming that it proactively constructs the prerequisite evidence (base facts) for all defined logical goals (inferred facts).

Table 3. Comparison of topological dataset statistics across generation methodologies. While the overall generation budgets (base+inferred targets) are matched to ∼640k triples, and the context graph sizes (input facts) are equalized within a 10% margin via sample trimming, the internal distribution of base versus inferred targets inherently diverges due to the structural differences produced by forward versus backward chaining.

Dataset	Method	Avg #/KG		Input	Base Targets			Inferred Targets			Ratio
		Base	*Inf.*	*Facts*	*Total*	*Pos*	*Neg*	*Total*	*Pos*	*Neg*	$\hat{\rho}_{\pm}$
Family tree	UDM	187.9	532.8	225k	306k	159k	147k	333k	160k	173k	0.998
exp. 2	SYNTHOLOGY	172.3	532.8	206k	391k	207k	184k	249k	113k	136k	0.998
OWL2Bench	UDM	67.4	210	14.5k	18.8k	6.1k	12.7k	26.4k	16.7k	9.7k	1.020
exp. 3	SYNTHOLOGY	252	824	13.9k	27.7k	13.9k	13.8k	17.6k	9.0k	8.6k	1.020

Table 4. Comparison of computational time breakdown across generation methodologies. *Base Gen.* denotes the initial procedural or random ABox instantiation. *Inference* captures the Java-based forward-chaining materialization (Jena) or backward-chaining search (SYNTHOLOGY). *Overhead* includes BFS subgraph extraction, derivation-tracing for hop-depth computation, and negative sampling mechanics.

Dataset	Method	Base Gen.	Inference	Overhead	Total Time
Family tree	UDM	<1 min	26 min	2 min	30 min
exp. 2	SYNTHOLOGY	/	10 min	1 min	11 min
OWL2Bench ($u = 3$)	UDM	<1 min	<1 min	4 min	6 min
exp. 3	SYNTHOLOGY	/	2 min	<1 min	2 min

The influence of these structural differences on model performance is reflected in Table 1. On the SYNTHOLOGY-generated test set, RRN_{Syn} (trained on SYNTHOLOGY data) substantially outperforms RRN_{UDM}, the UDM-trained RRN. PR-AUC improves by 21.5pp, AUC-ROC by 0.23, and FPR drops by 19.5pp. To control for test-set bias, we separately evaluated both models on a UDM test set, too. This second test set reveals that RRN_{Syn}'s advantage is *partially* inflated by test-set familiarity.

On UDM data, PR-AUC converges nearly to the same value (42.9% vs. 41.8%), and AUC-ROC becomes almost identical for both models (0.643 for RRN_{UDM} vs. 0.628 for RRN_{Syn}). Yet crucially, the FPR advantage proves robust: on UDM data, RRN_{Syn} maintains substantially lower FPR (25.7% vs. 45.3% at hop 1 and 24.5% vs. 44.3% at hop 2), as can be seen in Table 2. The same table shows that on the UDM test set, RRN_{Syn} outperforms on triple accuracy at hop 1 (65.4% vs. 57.1%), where both models receive similar input context. At hop 2, RRN_{UDM} exhibits higher F1 (59.3% vs. 42.6%), but this is entirely due to over-prediction: as noted before, its FPR is 44.3%.

These findings indicate that the SYNTHOLOGY test set overstates the PR-AUC and AUC-ROC advantages due to structural familiarity, while the FPR advantage (reflecting the model's learned precision in rejecting near-miss negatives) generalizes to test data generated by a different system. Finally, we note that SYNTHOLOGY generates the dataset roughly three times faster than UDM at this matched scale (11 min vs. 30 min). Table 4 shows the per-stage breakdown.

6.3. Experiment 3: Generalization to Complex Ontologies

Hop Distribution. Unlike the *random* ABox generation used in Experiment 2, the UDM baseline in Experiment 3 utilizes the OWL2Bench Java generator to *procedurally* instantiate realistic university structures. The baseline therefore produces a naturally deep and highly connected ABox before any materialization occurs, giving the forward-chainer a stronger starting point. Despite this favorable setup, SYNTHOLOGY still produces a

deeper hop distribution. Figure 2b shows that it creates a heavier tail of deep inferences than the benchmark-seeded baseline, demonstrating its ability to construct complex inferred facts even on richer schemas.

Dataset Topology. Table 3 shows that the structural patterns observed in Experiment 2 persist at a larger scale. Both generators were constrained to a nearly identical input graph size (14.5k vs. 13.9k input facts) and produced an equivalent total target count (∼45k). The internal split, however, differs: UDM produces 26.4k inferred targets against 18.8k base targets, while SYNTHOLOGY produces a 17.6k vs. 27.7k split. Because the OWL2Bench generator provides a structurally rich ABox out-of-the-box, the subsequent forward-chaining triggers a wave of shallow rule executions, dominating UDM's inference budget. SYNTHOLOGY's backward-chaining instead allocates the budget to constructing the specific base evidence required to support its targeted proofs, resulting in a leaner set of inferred targets that carries a heavier deep-hop tail.

Scalability: the Reasoning Wall. A final question is how each generator scales with ontology size. UDM materializes *full* deductive closures, while SYNTHOLOGY samples N *independent* proof trees. This difference is precisely what makes SYNTHOLOGY attractive when one wants a fixed-size training set rather than every entailed fact.

UDM iteratively saturates the ABox via forward-chaining and must hold every derived fact in memory. Because OWL 2 RL reasoning can be mapped to Datalog [15], its materialization inherits Datalog's data complexity: in the worst case, the closure size scales polynomially as $O(n^v)$, where n is the number of individuals and v is the maximum number of distinct variables in any rule body [21]. SYNTHOLOGY, by contrast, constructs each KG independently, so the per-sample memory footprint is constant in n.

As Table 5 shows, at small scale ($u = 3$ on OWL2Bench, roughly 10k base facts) both approaches complete in comparable time. The gap becomes severe at large scale ($u = 14$, ∼664k base facts): Jena's engine exhausts approximately 112 GB of JVM heap (across 8 cluster nodes) after approximately 7.5 hours, failing to terminate under resource constraints. SYNTHOLOGY generates one million target triples over the same ontology in under 70 minutes on a *single* node of the same cluster, with memory usage bounded by the per-sample proof-depth cap rather than cumulative closure size.

Table 5. The *reasoning wall*. At large scale, UDM's forward-chaining approach fails to terminate, while SYNTHOLOGY's independent backward-chaining scales linearly in the number of requested samples.

Scale	UDM Baseline (Jena)	SYNTHOLOGY
$u = 3$	6 min	2 min
$u = 14$	**OOM** after 7.5 h	70 min (1M targets)
Memory	polynomial in n cumulative	constant in n per-sample

We conclude that the SYNTHOLOGY pipeline scales to a more complex ontology without modification, that the deep-hop structural advantage of targeted generation is preserved at this larger scale, and that SYNTHOLOGY avoids the *reasoning wall* that the UDM baseline hits at $u = 14$. The takeaway is not that backward-chaining is universally faster than forward-chaining, but that for the practical task of *producing a fixed-size training set for a neurosymbolic reasoner*, sampling proofs is a fundamentally better fit than materializing the full closure.

Taken together, Experiment 3 demonstrates that SYNTHOLOGY preserves its structural advantages (deeper hop distribution, complete coverage) on a richer benchmark ontology while scaling past the point at which forward-chaining materialization fails. Critically, it does so with no ontology-specific engineering: the same backward-chaining pipeline that handled the family tree handled OWL2Bench. This is what restoring the symbolic-reasoner contract looks like in practice.

Generalization Beyond the RRN. Multi-hop coverage by construction, proof-based near-miss negatives, complete predicate coverage, and per-sample bounded memory are all evidenced directly through dataset-level metrics in Sections 6.2 and 6.3 and hold regardless of which model is trained on the data. We selected the RRN as the evaluation architecture precisely because it is explicitly tailored to multi-hop inference. A model that already specializes in multi-hop reasoning is the one for which the marginal value of multi-hop training data is *hardest* to demonstrate. Architectures with weaker inductive biases for multi-hop chains would, if anything, be expected to benefit *more* from such data, not less. Confirming this empirically across a broader range of neurosymbolic architectures is therefore left as a direction for future work.

7. Conclusion & Future Work Directions

Symbolic reasoners offer a simple contract: provide an ontology, receive entailments. Neurosymbolic reasoners break this contract by requiring a training dataset that, for most ontologies, does not exist, and existing synthetic pipelines produce such data only incidentally. We presented SYNTHOLOGY, an ontology-agnostic synthetic data generator that engineers training data by construction. Targeted backward-chaining proof search guarantees that every positive target carries a deep proof, and proof-based negative sampling derives near-miss negatives by perturbing a single leaf of one of those proofs. Across two ontologies, SYNTHOLOGY delivers consistent gains over UDM baselines, complete predicate coverage where UDM leaves entire relation families uninferred, and tractable generation at scales where forward-chaining materialization fails to terminate.

This work can be extended in serveral directions: using SYNTHOLOGY's proof-tree machinery for benchmarking neurosymbolic reasoners with controlled hop depth and calibrated negatives, evaluating its data on architectures beyond the RRN and on a wider range of ontologies, extending parser coverage to remaining OWL 2 RL constructs and other OWL 2 profiles, and exploring graduated-curriculum strategies for combining negative-sampling signals. These extensions notwithstanding, SYNTHOLOGY is already a step toward making neurosymbolic reasoners as readily deployable on a new ontology as their symbolic counterparts.

Declaration on Generative AI

During the preparation of this manuscript, the authors used GitHub Copilot, Google Gemini and Claude by Anthropic in order to: automate cumbersome (coding) tasks, assist in their Integrated Development Environment (IDE), help with paraphrasing and rewording of the paper. After using these tools/services, the authors have reviewed and edited the content as needed and take full responsibility for the publication's content.

References

[1] He T, Chen Z, Liao L, Cao Y, Liu Y, Tang W, et al. Subgraph-Centric Multi-Agent Reinforcement Learning for Multi-Hop Knowledge Graph Reasoning. IEEE Transactions on Knowledge and Data Engineering. 2026;38(2):1319-33.

[2] Cheng K, Ahmed NK, Rossi RA, Willke T, Sun Y. Neural-Symbolic Methods for Knowledge Graph Reasoning: A Survey. ACM Trans Knowl Discov Data. 2024 Nov;18(9). Available from: https://doi.org/10.1145/3686806.

[3] Nenov Y, Piro R, Motik B, Horrocks I, Wu Z, Banerjee J. RDFox: A highly-scalable RDF store. In: The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II 14. Springer; 2015. p. 3-20.

[4] Carroll JJ, Dickinson I, Dollin C, Reynolds D, Seaborne A, Wilkinson K. Jena: implementing the semantic web recommendations. In: Proceedings of the 13th international conference on World Wide Web; 2004. p. 74-83.

[5] Bonte P, Ongenae F, De Turck F. Subset reasoning for event-based systems. Ieee Access. 2019;7:107533-49.

[6] DeLong LN, Mir RF, Fleuriot JD. Neurosymbolic AI for Reasoning Over Knowledge Graphs: A Survey. IEEE Transactions on Neural Networks and Learning Systems. 2024;36(5):7822-42.

[7] Zhu X, Liu B, Zhu C, Ding Z, Yao L. Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning. Mathematics. 2023;11(3). Available from: https://www.mdpi.com/2227-7390/11/3/495.

[8] Auer S, Bizer C, Kobilarov G, Lehmann J, Cyganiak R, Ives Z. DBpedia: A nucleus for a web of open data. In: The Semantic Web: 6th International Semantic Web Conference, 2nd Asian Semantic Web Conference, ISWC 2007+ ASWC 2007, Busan, Korea, November 11-15, 2007. Proceedings. Springer; 2007. p. 722-35.

[9] Bollacker K, Evans C, Paritosh P, Sturge T, Taylor J. Freebase: a collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD international conference on Management of data; 2008. p. 1247-50.

[10] Guo Y, Pan Z, Heflin J. LUBM: A benchmark for OWL knowledge base systems. Journal of Web Semantics. 2005;3(2-3):158-82.

[11] Ma L, Yang Y, Qiu Z, Deforeit G, Wang C, Tommila A, et al. UOBM: A user-centric ontology benchmark for OWL. In: Asian Semantic Web Conference. Springer; 2006. p. 170-84.

[12] Anke LE, Declerck T, Gromann D, Makni B, Hendler J, Gromann D, et al. Deep learning for noise-tolerant RDFS reasoning1. Semant Web. 2019 Jan;10(5):823–862. Available from: https://doi.org/10.3233/SW-190363.

[13] Madushanka T, Ichise R. Negative Sampling in Knowledge Graph Representation Learning: A Review; 2024. Available from: https://arxiv.org/abs/2402.19195.

[14] Hohenecker P, Lukasiewicz T. Ontology reasoning with deep neural networks. Journal of Artificial Intelligence Research. 2020;68:503-40.

[15] Motik B, Grau BC, Horrocks I, Wu Z, Fokoue A, Lutz C. OWL 2 Web Ontology Language Profiles. W3C Recommendation; 2009. Available from: https://www.w3.org/TR/OWL~2-profiles/.

[16] Bordes A, Usunier N, Garcia-Duran A, Weston J, Yakhnenko O. Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems. vol. 26; 2013. .

[17] Singh G, Bhatia S, Mutharaju R. OWL2Bench: a benchmark for OWL 2 reasoners. In: International semantic web conference. Springer; 2020. p. 81-96.

[18] Geirhos R, Jacobsen JH, Michaelis C, Zemel RS, Bethge M, Wichmann FA. Shortcut Learning in Deep Neural Networks. Nature Machine Intelligence. 2020;2(11):665-73.

[19] Kommineni VK, König-Ries B, Samuel S. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction; 2024. Available from: https://arxiv.org/abs/2403.08345.

[20] Davis J, Goadrich M. The relationship between Precision-Recall and ROC curves. In: Proceedings of the 23rd International Conference on Machine Learning. ICML '06. New York, NY, USA: Association for Computing Machinery; 2006. p. 233–240. Available from: https://doi.org/10.1145/1143844.1143874.

[21] Dantsin E, Eiter T, Gottlob G, Voronkov A. Complexity and expressive power of logic programming. ACM Computing Surveys (CSUR). 2001;33(3):374-425.